# Lecture 41 Practice Solutions: Deep Learning (Part-6)

## Complete Mathematical Derivations & Code Solutions

---

### Solution 1: Exploratory Data Analysis & Deduplication

1. **Data Leakage Explanation:** Data leakage occurs when information from outside the training dataset is inappropriately used to train the machine learning model. If duplicate reviews are not purged prior to splitting, identical text strings will appear simultaneously in both the training set and the test set. The model can achieve near $100\%$ accuracy on those leaked test samples merely by memorizing specific phrasing rather than learning semantic sentiment patterns, yielding deceitfully high test metrics.
2. **Deduplication Numbers:**
    - Initial count = $50,000$ samples.
    - Duplicates dropped = $418$.
    - Clean dataset count = $50,000 - 418 = \mathbf{49,582}$ reviews.
    - Distribution after deduplication: Negative = $24,698$ ($49.81\%$), Positive = $24,884$ ($50.19\%$). The class balance remains virtually a perfect $50/50$ split.

---

### Solution 2: Lowercasing & Case Folding

1. **Necessity of Case Normalization:** In natural language corpora, word capitalization indicates sentence beginnings, proper nouns, or user emphasis, but rarely alters core semantic polarity. Case normalization projects all character representations into a unified lowercase manifold, preventing identical semantic tokens from being split across multiple vocabulary dimensions.
2. **Impact on Example Sentence:** `"The director did a Good job, but GOOD was not enough."`
    - **Without lowercasing:** The vocabulary registers two distinct features: `"Good"` and `"GOOD"`. Each feature receives a separate count of 1.
    - **With lowercasing:** Both collapse into the single token `"good"` with a count of 2.

- **Consequence:** Vocabulary size shrinks by 1, and the statistical frequency of `"good"` is doubled, strengthening the feature's signal-to-noise ratio.

---

## Solution 3: Regex URL Removal

1. **Syntax Breakdown of `re.sub(r"http\S+", "", text)`:**
    - `http`: Literal character matching for "http" (matches both "http://" and "https://").
    - `\S`: Special regex character class matching any non-whitespace character (alphanumerics, slashes, punctuation, dots).
    - `+`: Greedy quantifier matching one or more continuous non-whitespace characters until the next whitespace (space, tab, newline) is encountered.
2. **Why Not `.*` or `\w+`:**
    - `\w+` only matches alphanumeric characters and underscores; it immediately halts at `:` or `/`, failing to strip URLs like `http://google.com`.
    - `.*` is greedy and matches **everything** until the end of the line! Using `re.sub(r"http.*", "", text)` would delete the URL and every single word that follows it in the entire review!

---

## Solution 4: Regex Greedy vs Non-Greedy HTML Tag Stripping

Raw text: `"<br />This was an incredible movie!<br />Loved the plot."`

1. **Non-Greedy Match (`<.*?>`):** The `?` tells the quantifier to match the **minimum** number of characters needed to reach `>`.
    - Match 1: `<br />` → stripped.
    - Match 2: `<br />` → stripped.
    - **Output:** `"This was an incredible movie!Loved the plot."` (Correctly preserves review text).
2. **Greedy Match (`<.*>`):** Without `?`, `*` matches the **maximum** number of characters possible. It starts at the first `<` before `br` and matches all characters all the way to the final `>` in the second `<br />`.
    - It matches: `<br />This was an incredible movie!<br />`
    - **Output:** `"Loved the plot."`
    - **Disaster:** The entire first sentence is completely obliterated!

---

### Solution 5: Punctuation Stripping via Caret Negation

1. **Caret Negation Operator:** Inside a character class `[...]`, placing the caret `^` as the very first character negates the class: it matches any character that is **NOT** present in the specified list.
2. **Application to Example:** Raw: `"Wow!! The actor's performance was 10/10 #CinematicMasterpiece :)"`
    - Keeps: `A-Z`, `a-z`, `0-9`, and spaces (`\s`).
    - Strips: `!!`, `'`, `/`, `#`, `:`, `)`.
    - **Result:** `"Wow The actors performance was 1010 CinematicMasterpiece "`

---

### Solution 6: Word Tokenization with NLTK Punkt

1. **Why `text.split(" ")` Fails:** Naive whitespace splitting cannot separate trailing punctuations from words (e.g., `"movie!"` becomes a single token including the exclamation mark), preventing it from matching `"movie"`. It also fails on contractions like `"didn't"`, keeping it as one token rather than separating auxiliary verb and negation.
2. **NLTK `word_tokenize()` Behavior:**
    - `"didn't"` is intelligently split into `["did", "n't"]`, isolating the negation morpheme `n't` for sentiment analysis.
    - `"movie!"` is split into `["movie", "!"]`, isolating the clean word token.

---

### Solution 7: Stopwords Removal & Zipf's Law

1. **Zipf's Law Formula:**

$$f(w) \propto \frac{1}{\text{rank}(w)}$$

The frequency of a word is inversely proportional to its rank. Rank 1 word ("the") appears $\approx 7\%$ of the time; Rank 2 ("of") appears $\approx 3.5\%$; Rank 3 ("and") appears $\approx 2.3\%$. Summing the harmonic series $\sum_{r=1}^{100} \frac{1}{r}$ demonstrates that the top 100 functional words account for $\approx 50\%$ of all tokens in English text.

2. **Zero Discriminative Sentiment Value:** In sentiment analysis, whether a movie is rated 10/10 or 1/10, the author uses "the", "is", "a", "it", and "to" with nearly identical frequency distributions. These words have near-zero mutual information with class

label $y \in \{0, 1\}$. Removing them reduces sequence length by $> 50\%$ without removing any emotional polarity signals.

---

## Solution 8: Morphological Stemming vs Lemmatization

1. **Operational Differences:**
    - **Stemmer (Porter):** Applies crude, rule-based algorithmic heuristic string slicing (suffix stripping). Stems are often invalid dictionary strings ( `"wonder"` , `"techniq"` ).
    - **Lemmatizer (WordNet):** Consults a comprehensive morphological lexical database and parses Part-of-Speech (POS) context to return the true canonical base lemma ( `"technique"` , `"good"` ).
2. **Failure Modes:**
    - **Over-Stemming:** Collapsing distinct semantic words to the same stem (e.g., `"universal"` , `"universe"` , `"university"` all chopped to `"univers"` ).
    - **Under-Stemming:** Failing to collapse words of the same root (e.g., `"alumnus"` , `"alumni"` , `"alumnae"` remaining distinct).
3. **Why Stemming was Chosen:** Processing $49,582$ reviews involves over $1.5$ million words. WordNet Lemmatization requires Part-of-Speech tagging every sentence, requiring $> 15$ minutes of compute. The Porter Stemmer completes in under $45$ seconds and successfully conflates $> 92\%$ of sentiment inflection variants.

---

## Solution 9: TF-IDF Dimensionality & Memory Constraints

1. **Memory Calculation:** For $M = 49,582$ documents using 32-bit floats ($4$ bytes per float):
    - **Unconstrained ($V = 60,000$):**

$$\text{Bytes} = 49,582 \times 60,000 \times 4 = 11,899,680,000 \text{ bytes} \approx$$

    - **Constrained ( `max_features=5000` ):**

$$\text{Bytes} = 49,582 \times 5,000 \times 4 = 991,640,000 \text{ bytes} \approx \mathbf{945.}$$

    Setting `max_features=5000` achieves a $92\%$ **reduction in memory**, enabling smooth model training on standard hardware.

2. **Smooth IDF Formula:**

$$\text{IDF}(t) = \log\left(\frac{1+N}{1+\text{DF}(t)}\right) + 1$$

When a word appears in every document ($\text{DF}(t) = N$):

$$\text{IDF}(t) = \log\left(\frac{1+N}{1+N}\right) + 1 = \log(1) + 1 = 0 + 1 = \mathbf{1.0}$$

Its IDF weight is minimized to $1.0$, heavily suppressing its impact after $L_2$ normalization compared to rare keywords whose IDF can exceed $10.0$.

---

## Solution 10: PyTorch DataLoader & Shuffling Dynamics

1. **Batch Count & Final Batch Size:**
   - Total train samples = $39,665$. Batch size = $64$.
   - Total batches per epoch:

$$\text{Batches} = \left\lceil \frac{39,665}{64} \right\rceil = \mathbf{620} \text{ batches}$$

   - Full batches = $619 \times 64 = 39,616$ samples.
   - Final batch sample count = $39,665 - 39,616 = \mathbf{49}$ samples.
2. **Statistical Violation of `shuffle=False`:** Stochastic Gradient Descent assumes that each mini-batch gradient $\nabla \mathcal{L}_{\text{batch}}$ is an unbiased estimator of the true population gradient $\nabla \mathcal{L}$. Without shuffling, if reviews are sorted or clustered by author or rating, mini-batch gradients become correlated, causing parameter updates to oscillate erratically and stall in poor local minima.

---

## Solution 11: 3D Input Tensor Reshaping for PyTorch RNN

1. **Shape After Unsqueeze:** `Xb` of shape `(64, 5000)` passed through `Xb.unsqueeze(1)` becomes:

$$\mathbf{X}_{\text{reshaped}} \in \mathbb{R}^{\mathbf{64 \times 1 \times 5000}}$$

2. **Dimensions Meaning:**

- **Dimension 0 ( `64` ):** Batch Size (number of independent reviews processed in parallel).
- **Dimension 1 ( `1` ):** Sequence Length $T = 1$. Because the entire document is summarized as a single 5000-dimensional TF-IDF vector, the temporal length is 1 time-step.
- **Dimension 2 ( `5000` ):** Input Feature Dimension (the 5000 TF-IDF word weights).

---

## Solution 12: Exact Parameter Count Derivation for PyTorch RNN

1. **Recurrent Layer ( `nn.RNN(5000, 128)` ):**
   - $\mathbf{W}_{xh} \in \mathbb{R}^{128 \times 5000} \implies 128 \times 5,000 = 640,000$ weights.
   - $\mathbf{W}_{hh} \in \mathbb{R}^{128 \times 128} \implies 128 \times 128 = 16,384$ weights.
   - $\mathbf{b}_h \in \mathbb{R}^{128} \implies 128$ biases.

$$\text{Subtotal RNN} = 640,000 + 16,384 + 128 = \mathbf{656,512} \text{ par}$$

2. **Linear Layer ( `nn.Linear(128, 1)` ):**
   - $\mathbf{W}_{hy} \in \mathbb{R}^{1 \times 128} \implies 1 \times 128 = 128$ weights.
   - $b_y \in \mathbb{R}^{1} \implies 1$ bias.

$$\text{Subtotal Linear} = 128 + 1 = \mathbf{129} \text{ parameters}$$

3. **Total Model Parameters:**

$$\text{Total} = 656,512 + 129 = \mathbf{656,641} \text{ learnable parameters}$$

---

## Solution 13: Binary Cross-Entropy Loss & Sigmoid Activation

1. **Mini-Batch Loss Formula:**

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \left[ y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

2. **Loss Comparison for $y = 1$:**
   - When $\hat{p} = 0.99$:

$$\text{Loss} = -\ln(0.99) \approx \mathbf{0.01005}$$

- When $\hat{p} = 0.01$:

$$\text{Loss} = -\ln(0.01) \approx \mathbf{4.60517}$$

- **Analysis:** The loss for the incorrect prediction is over $458\times$ **larger**! Binary Cross-Entropy imposes severe, exponential penalties on confident incorrect predictions.

---

## Solution 14: PyTorch Model Evaluation Protocol

1. `model.eval()` **Effects:**
   - Freezes Dropout layers (dropout probability set to 0; all neurons remain active).
   - Freezes Batch Normalization running statistics (uses running mean and variance instead of mini-batch statistics).
2. `with torch.no_grad()` **Technical Impact:**
   - Halts the PyTorch Autograd execution engine. No computation graph nodes are recorded in memory. This eliminates intermediate activation caching, saving $> 60\%$ of RAM/VRAM and accelerating inference by $> 2\times$.
3. **Probability & Prediction for $z = -1.386$:**

$$\hat{p} = \sigma(-1.386) = \frac{1}{1 + e^{1.386}} = \frac{1}{1 + 3.9988} = \frac{1}{4.9988} \approx \mathbf{0.200}$$

Since $\hat{p} = 0.20 \leq 0.5$, the predicted class is $\hat{y} = 0$ **(Negative Sentiment)**.

---

## Solution 15: Comparative Paradigm Study: CNN vs RNN on MNIST

1. **Formulating 2D Image as 1D Sequence for RNN:** An MNIST image has $28 \times 28$ grayscale pixels. To process it sequentially:
   - Sequence Length $T = 28$ time steps (corresponding to the 28 image rows from top to bottom).
   - Input Feature Dimension $d_x = 28$ (the 28 pixel values comprising row $t$).
   - At step $t = 1$, the RNN reads Row 1; at step $t = 28$, it reads Row 28 and predicts the digit from final hidden state $\mathbf{h}_{28}$.

2. **Why CNN Outperforms RNN on Images:**
    - **CNN Inductive Bias:** CNNs encode **2D spatial locality** and **translation equivariance** via sliding 2D convolution kernels ($3 \times 3$). Nearby pixels along both rows and columns are processed together with weight sharing, perfectly matching visual structure.
    - **RNN Inductive Bias:** RNNs assume **1D directional temporal causality**. Vertical spatial relationships between Row 2 and Row 26 must be remembered across 24 recurrence loops through a finite hidden state vector, suffering from attenuation.
    - **Result:** CNN achieves $> 99.2\%$ with fewer parameters, while RNN caps out at $\approx 96.5\% - 98.0\%$.